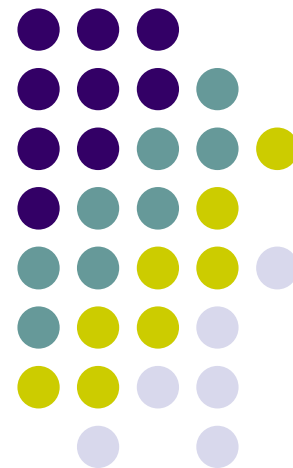


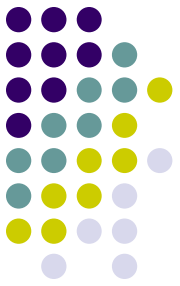
第8章 NP完全性理论



学习要点

- 理解RAM, RASP和图灵机计算模型
- 理解非确定性图灵机的概念
- 理解P类与NP类语言的概念
- 理解NP完全问题的概念





计算复杂度

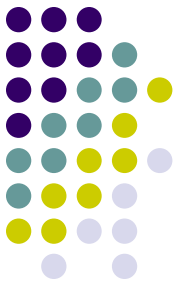
- 研究计算问题时所需的资源
 - 时间（要通过多少步才能解决问题）
 - 空间（在解决问题时需要多少内存）



时间复杂度

$$O(\log n) < O(n) < O(n \log_2 n) < O(n^2) \\ < O(n^k) < O(2^n) < O(k^n) < O(n!)$$

n	$\log n$	n	$n \log n$	n^2	n^3	2^n	3^n	$n!$
10	10^{-6}	10^{-5}	10^{-5}	10^{-4}	10^{-3}	10^{-3}	0.059	0.45
20	10^{-6}	10^{-5}	10^{-5}	10^{-4}	10^{-2}	1(秒)	58(分)	1年
50	10^{-5}	10^{-4}	10^{-4}	0.0025	0.125	36年	2×10^{10} 年	10^{57} 年
1000	10^{-5}	10^{-3}	10^{-3}	1	16小時	10^{333} 年	極大	極大
10^6	10^{-5}	1	6	1月	10^5 年	極大	極大	極大
10^9	10^{-5}	16小時	6天	3年	3×10^9 年	極大	極大	極大

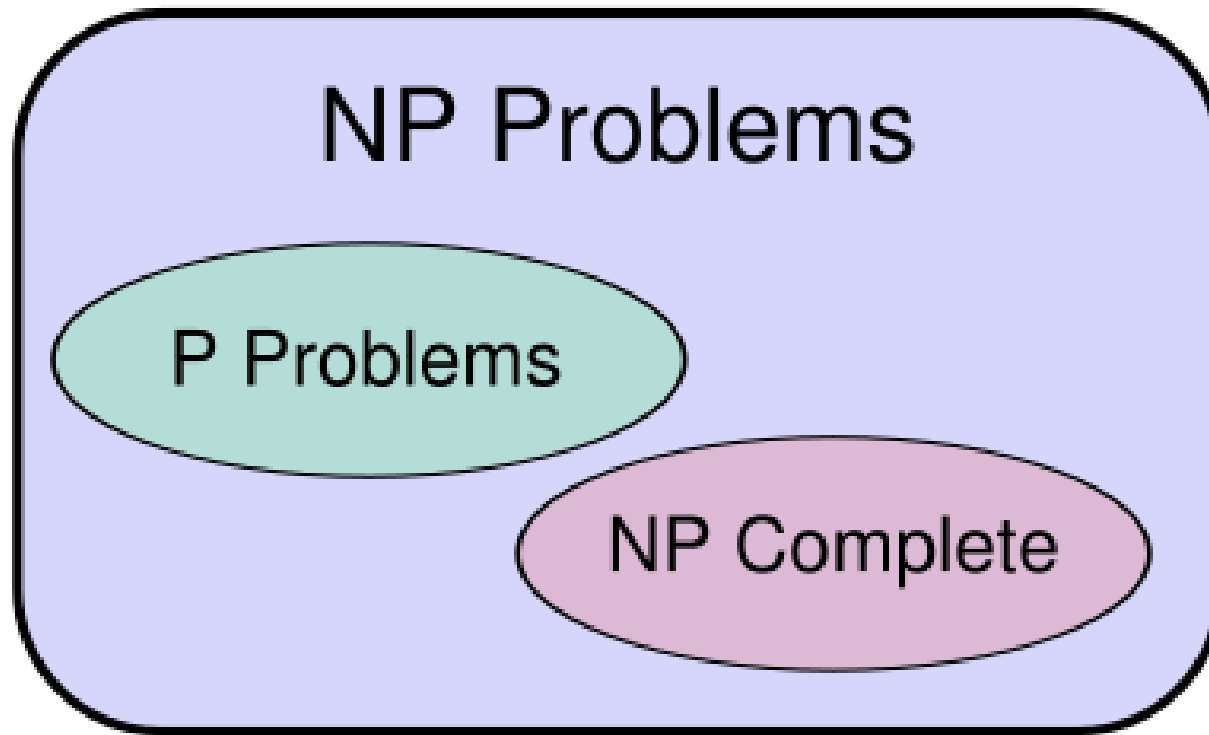
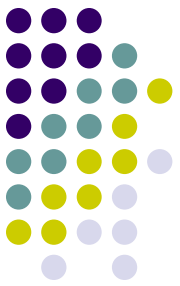


问题

- 何为**P**问题？
- 何为**NP**问题？
- 何为**NPC**问题？
- 何为**NP HARD**问题？

- 为什么研究**NP**问题：
 - 回答“求解的问题到底有多复杂”

- 通常认为，在多项式时间内可解的问题称为“易”解问题；需要指数时间求解的问题为“难”解问题。

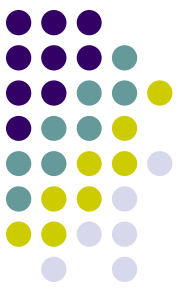


- 多项式时间算法: **Polynomial-time algorithms**

对于规模为 n 的输入, 它们在最坏的情况下的运行时间为 $O(n^k)$, 其中 k 为某个常数。

- 指数时间算法

$$O(\log n) < O(n) < O(n \log_2 n) < O(n^2) \\ < O(n^k) < O(2^n) < O(k^n) < O(n!)$$



8.1 计算模型

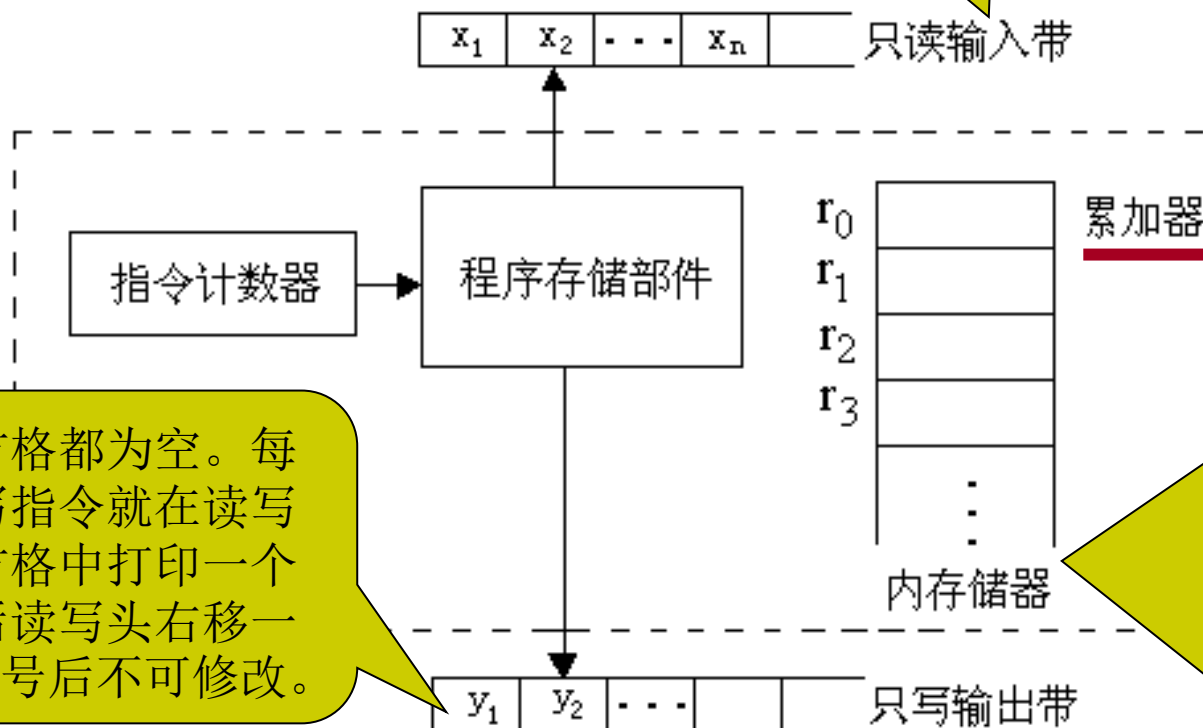
- 在进行问题的计算复杂性分析之前，首先必须建立求解问题所用的计算模型，包括定义该计算模型中所用的基本运算。
- 建立计算模型的目的是为了使问题的计算复杂性分析有一个共同的客观尺度。
- 3个基本计算模型：
 - 随机存取机RAM(Random Access Machine);
 - 随机存取存储程序机RASP(Random Access Stored Program Machine)
 - 图灵机(Turing Machine)。
- 这3个计算模型在计算能力上是等价的，但在计算速度上是不同的。

RAM是一台单累加器计算机



8.1.1 随机存取机RAM

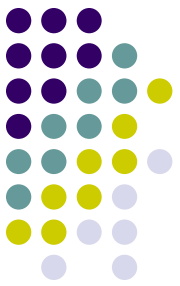
1、RAM的结构



由一组方格组成，每个方格可存放一个整数。每从只读输入带上读入一个数后，读写头右移一格。

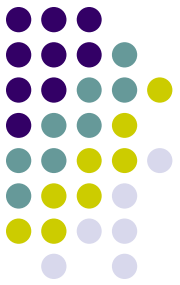
由一组寄存器 $r_1, r_2, \dots, r_i, \dots$ 组成。每个寄存器可以存放一个整数，且存储器的个数不限。这说明当所求解问题的规模不超过一台计算机的内存容量，且整数字长不超过计算机字长时，RAM模型的抽象是符合实际的。

初始每个方格都为空。每执行一条写指令就在读写头下方的方格中打印一个整数，然后读写头右移一格。输出符号后不可修改。



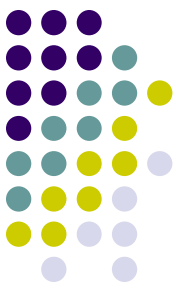
随机存取机RAM

- **RAM程序**是带标号的指令序列，存储在程序存储部件中，而非内部存储器中。
- RAM设有算术运算指令，输入输出指令，存取数指令及转移指令。
- **RAM指令**由操作码和操作数组成。其中，操作数有3种形式：
 - a) $=i$ ，操作数是整数 i 本身。（直接数型）
 - b) i ， i 是非负整数，操作数是寄存器 r_i 的内容。（直接地址型）
 - c) $*i$ ， i 为非负整数，若寄存器 r_i 的内容为整数 j ，则操作数为寄存器 r_j 中的内容。（间接地址型）



随机存取机RAM

- RAM对所有无定义的指令作停机处理。
- 设 c 是内存映射函数，即 $c(i)$ 表示寄存器 r_i 中的内容。则上述三种类型操作数的值 V 分别为：
 - $V(=i)=i$;
 - $V(i)=c(i)$;
 - $V(*i)=c(c(i))$;



RAM基本指令集

操作码	操作数	指令含义
(1) LOAD	$= i / i / * i$	取操作数入累加器
(2) STORE	$i / * i$	将累加器中数存入内存
(3) ADD	$= i / i / * i$	加法运算
(4) SUB	$= i / i / * i$	减法运算
(5) MULT	$= i / i / * i$	乘法运算
(6) DIV	$= i / i / * i$	除法运算
(7) READ	$i / * i$	读入
(8) WRITE	$= i / i / * i$	输出
(9) JUMP	标号	无条件转移到标号语句
(10) JGTZ	标号	正转移到标号语句
(11) JZERO	标号	零转移到标号语句
(12) HALT		停机



8.1.1 随机存取机RAM

2、RAM程序

一个RAM程序定义了从输入带到输出带的一个映射。可以对这种映射关系作2种不同的解释。

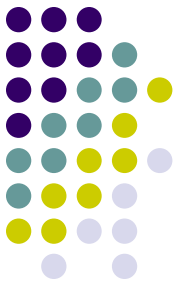
解释一：把RAM程序看成是计算一个函数

若一个RAM程序P总是从输入带前 n 个方格中读入 n 个整数 $x_1, x_2, \dots, x_n$ ，并且在输出带的第一个方格上输出一个整数 y 后停机，那么就说**程序P计算了函数** $f(x_1, x_2, \dots, x_n)=y$

解释二：把RAM程序当作一个语言接受器。

将字符串 $S=a_1a_2\dots a_n$ 放在输入带上。在输入带的第一个方格中放入符号 a_1 ，第二个方格中放入符号 a_2 ， $\dots$ ，第 n 个方格中放入符号 a_n 。然后在第 $n+1$ 个方格中放入0，作为输入串的结束标志符。如果一个RAM程序P读了字符串 S 及结束标志符0后，在输出带的第一格输出一个1并停机，就说**程序P接受字符串S**。

RAM程序P可接受的语言是指P可接受的所有字符串的集合 12



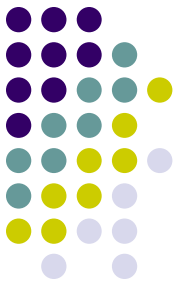
例子一

- 定义函数

$$f(n) = \begin{cases} n^n & n > 0 \\ 0 & n \leq 0 \end{cases}$$

相应的RAM程序见P282

```
int f(int r1)
{
    if(r1<=0) return 0;
    int r2=r1;
    int r3=r1-1;
    while(r3>0){
        r2=r2*r1;
        r3=r3-1;
    }
    return r2;
}
```



例子二

- 接受字母表{1,2}上的一个语言。
- 该语言包含所有由同样多个1和2组成的字符串。

相应的RAM程序见P283

```
int accept()
{
    int x;
    int d=0;
    x=readint();
    while(x!=0){
        if(x!=1) d--;
        else d++;
        x=readint();
    }
    if(d==0) return 1;
    else return 0;
}
```



8.1.1 随机存取机RAM

3、RAM程序的耗费标准

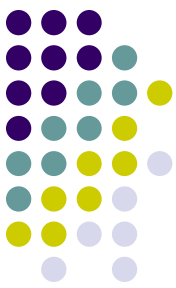
标准一：均匀耗费标准

在均匀耗费标准下，每条RAM指令需要一个单位时间；每个寄存器占用一个单位空间。以后除特别注明，RAM程序的复杂性将按照均匀耗费标准衡量。

标准二：对数耗费标准

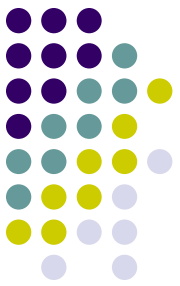
对数耗费标准是基于这样的假定，即执行一条指令的耗费与以二进制表示的指令的操作数长度成比例。在RAM计算模型下，假定一个寄存器可存放一个任意大小的整数。因此若设 $l(i)$ 是整数 i 所占的二进制位数，则

$$l(i) = \begin{cases} \lfloor \log |i| \rfloor & i \neq 0 \\ 1 & i = 0 \end{cases}$$



RAM 指令	对数耗费
(1) LOAD a	$t(a)$
(2) STORE i	$l(c(0)) + l(i)$
STORE $* i$	$l(c(0)) + l(i) + l(c(i))$
(3) ADD a	$l(c(0)) + t(a)$
(4) SUB a	$l(c(0)) + t(a)$
(5) MULT a	$l(c(0)) + t(a)$
(6) DIV a	$l(c(0)) + t(a)$
(7) READ i	$l(input) + l(i)$
READ $* i$	$l(input) + l(i) + l(c(i))$
(8) WRITE a	$t(a)$
(9) JUMP b	1
(10) JGTZ b	$l(c(0))$
(11) JZERO b	$l(c(0))$
(12) HALT	1

操作数 a	对数耗费 $t(a)$
$= i$	$l(i)$
i	$l(i) + l(c(i))$
$* i$	$l(i) + l(c(i)) + l(c(c(i)))$



8.1.2 随机存取存储程序机RASP

1、RASP的结构

RASP的整体结构类似于RAM，所不同的是RASP的程序是存储在寄存器中的。每条RASP指令占据2个连续的寄存器。第一个寄存器存放操作码的编码，第二个寄存器存放地址。RASP指令用整数进行编码。

指 令	编 码	指 令	编 码
LOAD i	1	DIV i	10
LOAD = i	2	DIV = i	11
STORE i	3	READ i	12
ADD i	4	WRITE i	13
ADD = i	5	WRITE = i	14
SUB i	6	JUMP i	15
SUB = i	7	JGTZ i	16
MULT i	8	JZERO i	17
MULT = i	9	HALT	18



随机存取存储程序机RASP

2、RASP程序的复杂性

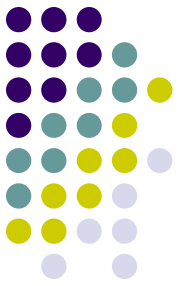
不管是在均匀耗费标准下，还是在对数耗费标准下，RAM程序和RASP程序的复杂性只差一个常数因子。在一个计算模型下 $T(n)$ 时间内完成的输入-输出映射可在另一个计算模型下模拟，并在 $kT(n)$ 时间内完成。其中 k 是一个常数因子。空间复杂性的情况也是类似的。

- 见书上例子：
 - 通过对RAM指令的模拟构造一个占据RASP的 $r-1$ 个寄存器的相应RASP程序 P_s 。
 - 模拟RAM命令SUB *i需要6条RASP指令。
 - 用RAM程序模拟RASP程序



随机存取存储程序机RASP

- 定理8.1
 - 不论在均匀耗费标准还是对数耗费标准下，对时间复杂性为 $T(n)$ 的任一RAM程序，都存在一个时间复杂性为 $kT(n)$ 的RASP程序与之等价，其中， k 为一常数。
- 定理8.2
 - 不论在均匀耗费标准还是对数耗费标准下，对时间复杂性为 $kT(n)$ 的任一RASP 程序，都存在一个时间复杂性为 $T(n)$ 的RAM程序与之等价，其中， k 为一常数。
- 从计算复杂性的观点看，**RAM计算模型和RASP计算模型在相差一个常数因子的意义下是等价的。**



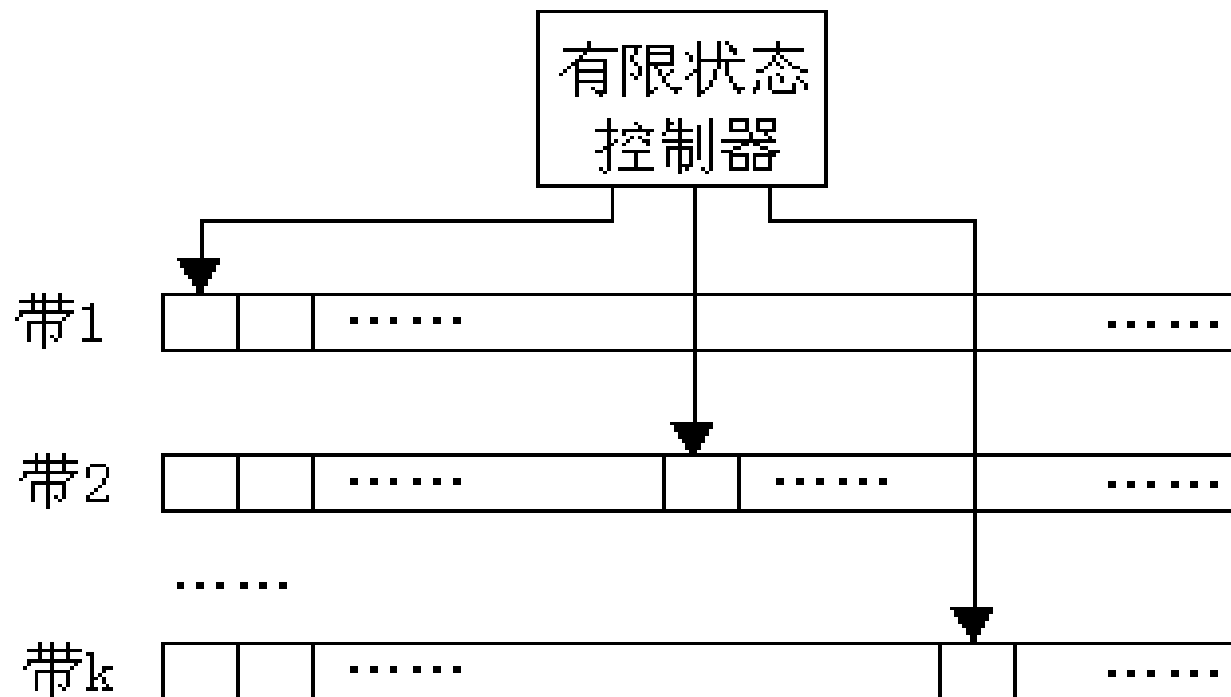
RAM的种种变形

- 实随机存取机RRAM
- 直线式程序(straight line programs)
- 位式计算(bitwise computation)
- 位向量运算(bit vector operations)
- 判定树
- 代数计算树(ACT)
- 代数判定树(ADT)

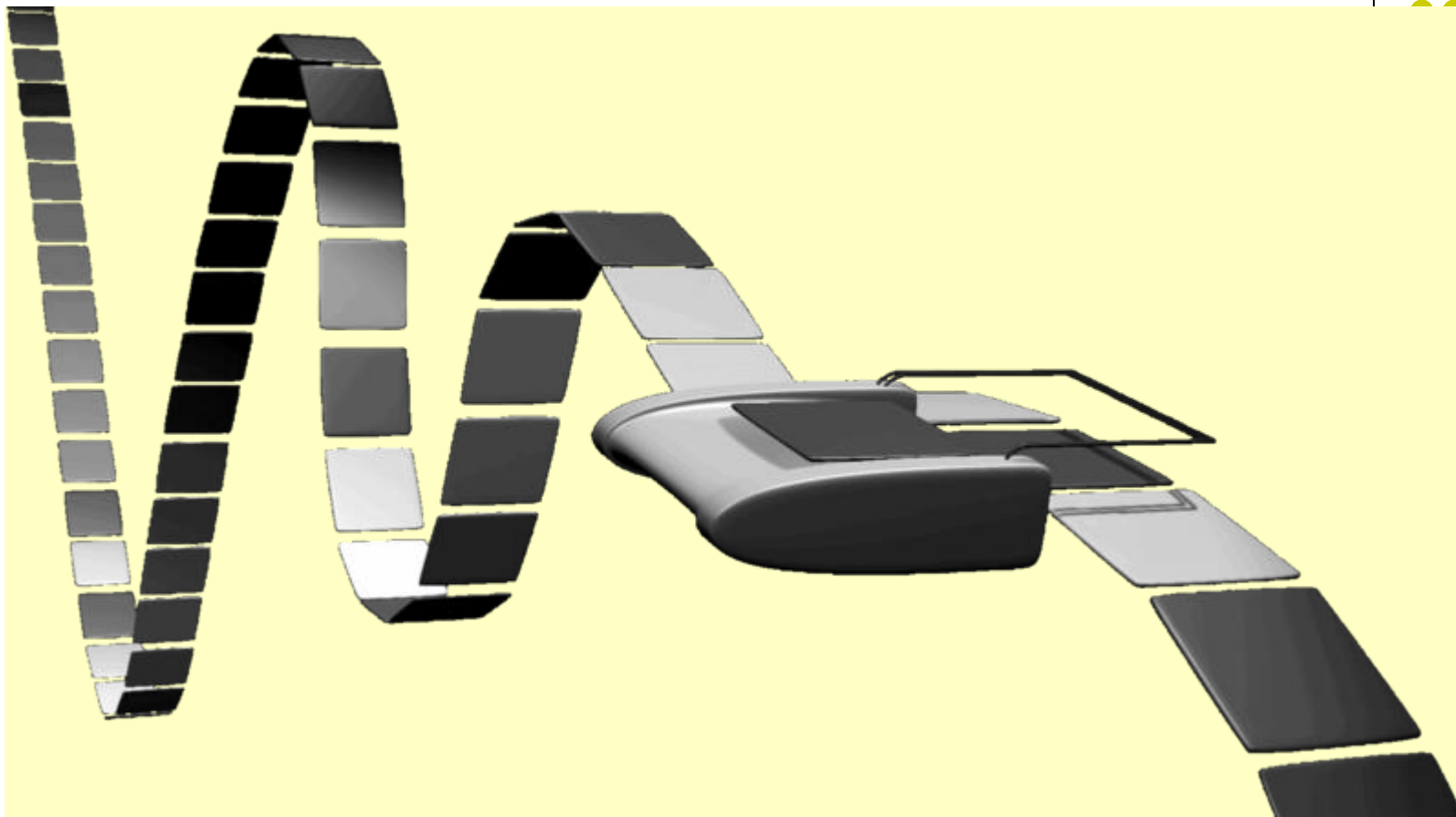


8.1.3 图灵机

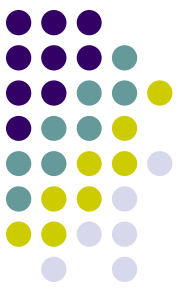
图灵机是一个结构简单且
计算能力很强的计算模型。



图灵机



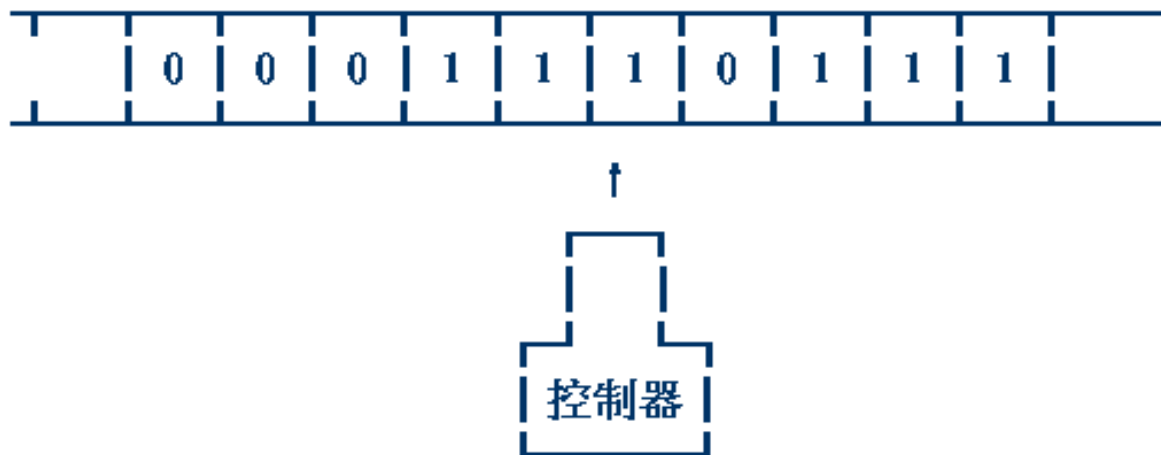
图灵机（英语：Turing Machine，又称确定型图灵机）是英国数学家阿兰·图灵于1936年提出的一种抽象计算模型，其更抽象的意义为一种数学逻辑机，可以看作等价于任何有限逻辑数学过程的终极强大逻辑机器。

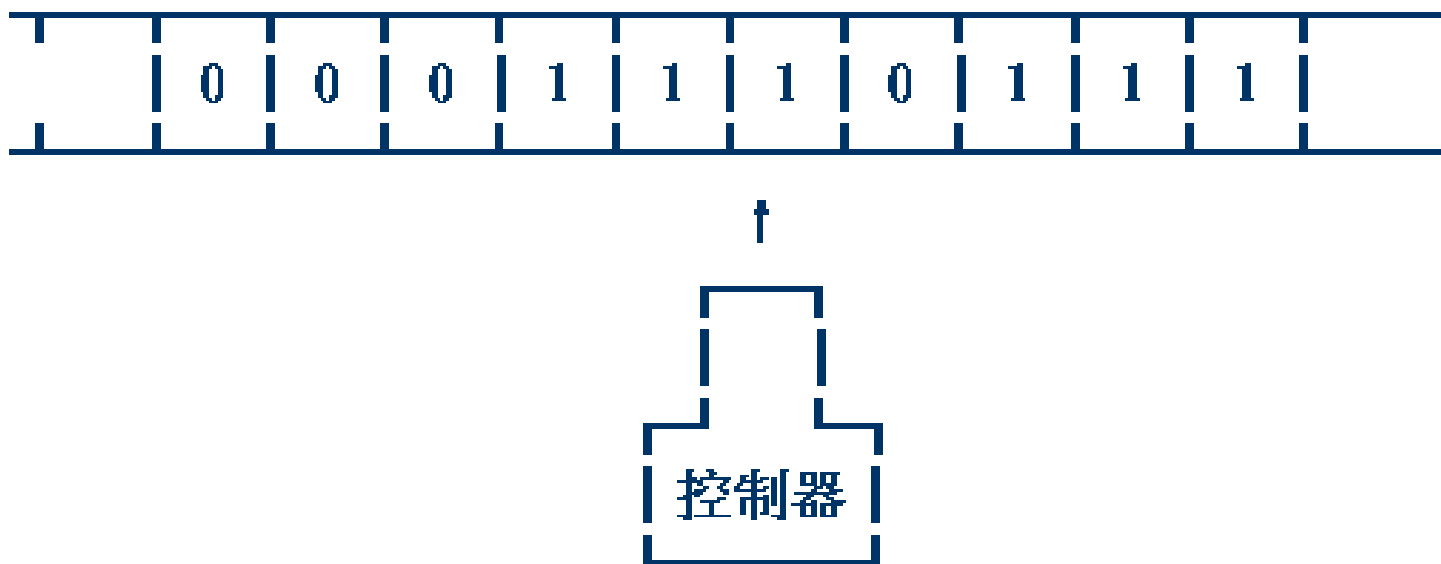


图灵的基本思想

- 图灵的基本思想是用机器来模拟人们用纸笔进行数学运算的过程，他把这样的过程看作下列两种简单的动作：
 1. 在纸上写上或擦除某个符号；
 2. 把注意力从纸的一个位置移动到另一个位置；
- 而在每个阶段，人要决定下一步的动作，依赖于 (a) 此人当前所关注的纸上某个位置的符号和(b) 此人当前思维的状态。

- 为了模拟人的这种运算过程，图灵构造出一台假想的机器，该机器由以下几个部分组成：
 1. 一条无限长的纸带 **TAPE**。纸带被划分为一个接一个的小格子，每个格子上包含一个来自有限字母表的符号，字母表中有一个特殊的符号表示空白。纸带上的格子从左到右依次被编号为 $0, 1, 2, \dots$ ，纸带的右端可以无限伸展。
 2. 一个读写头 **HEAD**。该读写头可以在纸带上左右移动，它能读出当前所指的格子上的符号，并能改变当前格子上的符号。
 3. 一套控制规则 **TABLE**。它根据当前机器所处的状态以及当前读写头所指的格子上的符号来确定读写头下一步的动作，并改变状态寄存器的值，令机器进入一个新的状态。
 4. 一个状态寄存器。它用来保存图灵机当前所处的状态。图灵机的所有可能状态的数目是有限的，并且有一个特殊的状态，称为停机状态。
- 注意这个机器的每一部分都是有限的，但它有一个潜在的无限长的纸带，因此这种机器只是一个理想的设备。图灵认为这样的一台机器就能模拟人类所能进行的任何计算过程。





- 一旦图灵机在运行中进入了一个状态，而且这个状态是一个结束状态，那么，图灵机就停机，计算任务宣告完成，此时带上的内容就是计算的输出结果。

8.1.3 图灵机



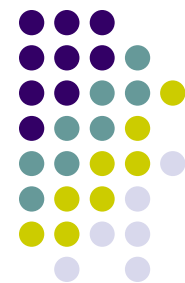
根据有限状态控制器的当前状态及每个读写头读到的带符号，图灵机的一个计算步可实现下面3个操作之一或全部。

- (1) 改变有限状态控制器中的状态。
- (2) 清除当前读写头下的方格中原有带符号并写上新的带符号。
- (3) 独立地将任何一个或所有读写头，向左移动一个方格(L)或向右移动一个方格(R)或停在当前单元不动(S)。

k带图灵机可形式化地描述为一个7元组 $(Q, T, I, \delta, b, q_0, q_f)$ ，其中：

- | | |
|-------------------------------------|----------------------------------|
| (1) Q 是有限个状态的集合。 | (2) T 是有限个带符号的集合。 |
| (3) I 是输入符号的集合， $I \subseteq T$ 。 | (4) b 是唯一的空白符， $b \in T - I$ 。 |
| (5) q_0 是初始状态。 | (6) q_f 是终止(或接受)状态。 |
| (7) δ 是移动函数。 | |

它是从 $Q \times T^k$ 的某一子集映射到 $Q \times (T \times \{L, R, S\})^k$ 的函数。

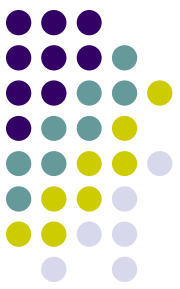


8.1.3 图灵机

与RAM模型类似，图灵机既可作为语言接受器，也可作为计算函数的装置。

图灵机M的时间复杂性 $T(n)$ 是它处理所有长度为 n 的输入所需的最大计算步数。如果对某个长度为 n 的输入，图灵机不停机， $T(n)$ 对这个 n 值无定义。

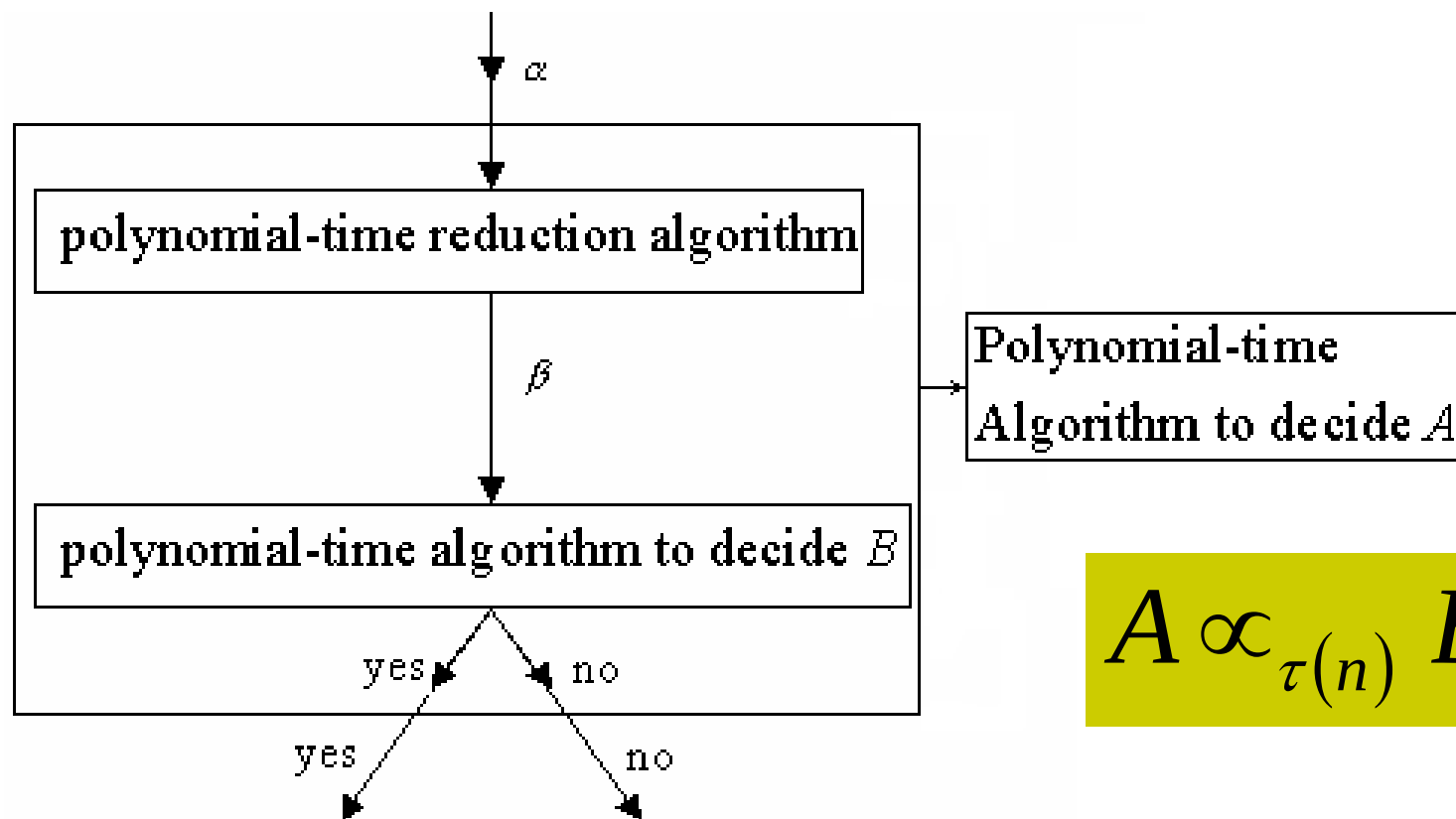
图灵机的空间复杂性 $S(n)$ 是它处理所有长度为 n 的输入时，在 k 条带上所使用过的方格数的总和。如果某个读写头无限地向右移动而不停机， $S(n)$ 也无定义。



8.1.5 图灵机模型与RAM模型的关系

- 定理8.3
 - 对于问题P的任何长度为n的输入，设求解问题P的算法A在k带图灵机模型TM下的时间复杂性为 $T(n)$ ，那么，算法A在RAM模型下的时间复杂性为 $O(T^2(n))$ 。
- 定理8.4
 - 对于问题P的任何长度为n的输入，设求解问题P的算法A在RAM模型下的时间复杂性为 $T(n)$ ，那么，算法A在k带图灵机模型TM下的时间复杂性为 $O(T^2(n))$ 。
- 在对数耗费下的RAM模型和RASP模型是与图灵机模型多项式相关的计算模型。对空间复杂性也可得到类似的结论。

8.1.6 问题变换与计算复杂性归约

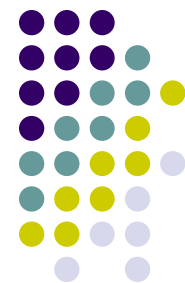


- 给定问题A的一个实例 α ，利用多项式时间归约算法，将它转换为问题B的一个实例 β 。
- 在实例 β 上，运行B的多项式时间判定算法。
- 将 β 的答案用作 α 的答案。



8.2 P类与NP类问题

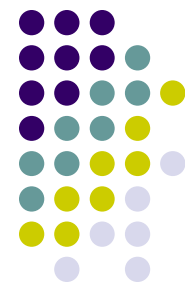
- 一般地说，将可由多项式时间算法求解的问题看作是易处理的问题，而将需要超多项式时间才能求解的问题看作是难处理的问题。
- 有许多问题，从表面上看似乎并不比排序或图的搜索等问题更困难，然而至今人们还没有找到解决这些问题的多项式时间算法，也没有人能够证明这些问题需要超多项式时间下界。
- 在图灵机计算模型下，这类问题的计算复杂性至今未知。
- 为了研究这类问题的计算复杂性，人们提出了另一个能力更强的计算模型，即非确定性图灵机计算模型，简记为NDTM(Nondeterministic Turing Machine)。
- 在非确定性图灵机计算模型下，许多问题可以在多项式时间内求解。



8.2.1 非确定性图灵机

在图灵机计算模型中，移动函数 δ 是单值的，即对于 $Q \times T^k$ 中的每一个值，当它属于 δ 的定义域时， $Q \times (T \times \{L, R, S\})^k$ 中只有唯一的值与之对应，称这种图灵机为**确定性图灵机**，简记为 **DTM** (Deterministic Turing Machine)。

非确定性图灵机 (NDTM)：一个 k 带的非确定性图灵机 M 是一个 7 元组： $(Q, T, I, \delta, b, q_0, q_f)$ 。与确定性图灵机不同的是非确定性图灵机允许移动函数 δ 具有**不确定性**，即对于 $Q \times T^k$ 中的每一个值 $(q; x_1, x_2, \dots, x_k)$ ，当它属于 δ 的定义域时， $Q \times (T \times \{L, R, S\})^k$ 中有唯一的一个子集 $\delta(q; x_1, x_2, \dots, x_k)$ 与之对应。可以在 $\delta(q; x_1, x_2, \dots, x_k)$ 中随意选定一个值作为它的函数值。



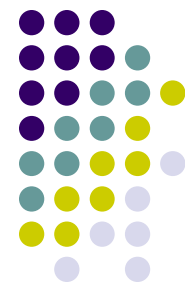
8.2.2 P类与NP类语言

P类和NP类语言的定义：

$P = \{L \mid L \text{ 是一个能在多项式时间内被一台DTM所接受的语言}\}$

$NP = \{L \mid L \text{ 是一个能在多项式时间内被一台NDTM所接受的语言}\}$

由于一台确定性图灵机可看作是非确定性图灵机的特例，所以可在多项式时间内被确定性图灵机接受的语言也可在多项式时间内被非确定性图灵机接受。故 $P \subseteq NP$ 。



8.2.2 P类与NP类语言

NP类语言举例——无向图的团问题。

该问题的输入是一个有 n 个顶点的无向图 $G=(V, E)$ 和一个整数 k 。要求判定图 G 是否包含一个 k 顶点的完全子图(团)，即判定是否存在 $V' \subseteq V$ ， $|V'|=k$ ，且对于所有的 $u, v \in V'$ ，有 $(u, v) \in E$ 。

若用邻接矩阵表示图 G ，用二进制串表示整数 k ，则团问题的一个实例可以用长度为 $n^2 + \log k + 1$ 的二进位串表示。因此，团问题可表示为语言：

$\text{CLIQUE} = \{w\#v \mid w, v \in \{0, 1\}^*, \text{以} w \text{为邻接矩阵的图} G \text{有一个} k \text{顶点的团, 其中} v \text{是} k \text{的二进制表示。}\}$

8.2.2 P类与NP类语言



接受该语言CLIQUE的**非确定性算法**：用非确定性选择指令选出包含 k 个顶点的候选顶点子集 V ，然后确定性地检查该子集是否是团问题的一个解。算法分为3个阶段：

算法的第一阶段将输入串 $w\#v$ 分解，并计算出 $n = \sqrt{|w|}$ ，以及用 v 表示的整数 k 。若输入不具有形式 $w\#v$ 或 $|w|$ 不是一个平方数就拒绝该输入。显而易见，第一阶段可 $O(n^2)$ 在时间内完成。

在算法的第二阶段中，非确定性地选择 V 的一个 k 元子集 $V' \subseteq V$ 。见书P303.

算法的第三阶段是确定性地检查 V' 的团性质。若 V' 是一个团则接受输入，否则拒绝输入。这可以在 $O(n^4)$ 时间内完成。因此，整个算法的时间复杂性为 $O(n^4)$

非确定性算法在多项式时间内接受语言CLIQUE，故 $\text{CLIQUE} \in \text{NP}$ 。

NP问题不是非P类问题。

NP问题是指可以在多项式的时间里验证一个解的问题。

8.2.3 多项式时间验证



多项式时间可验证语言类VP可定义为：

$VP = \{L \mid L \in \Sigma^*, \Sigma \text{ 为一有限字符集, 存在一个多项式 } p \text{ 和一个多项式时间验证算法 } A(X, Y) \text{ 使得对任意 } X \in \Sigma^*, X \in L \text{ 当且仅当存在 } Y \in \Sigma^*, |Y| \leq p(|X|) \text{ 且 } A(X, Y) = 1\}$ 。

定理8.5: $VP = NP$ 。

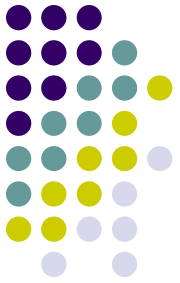
例如(哈密顿回路问题)：一个无向图G含有哈密顿回路吗？

无向图G的哈密顿回路是通过G的每个顶点恰好一次的简单回路。
可用语言HAM-CYCLE 定义该问题如下：

$HAM-CYCLE = \{G \mid G \text{ 含有哈密顿回路}\}$

NP否？

思考：试问一个图中是否不存在Hamilton回路？



8.3 NP完全问题

- 8.3.1 多项式时间变换
- 8.3.2 Cook定理

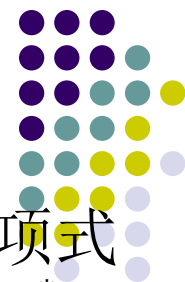


8.3 NP完全问题NPC

- $P \subseteq NP$ 。
- 直观上看，P类问题是确定性计算模型下的易解问题类，而NP类问题是非确定性计算模型下的易验证问题类。
- 大多数的计算机科学家认为NP类中包含了不属于P类的语言，即 $P \neq NP$ 。
- NP完全问题有一种令人惊奇的性质，即如果一个NP完全问题能在多项式时间内得到解决，那么NP中的每一个问题都可以在多项式时间内求解，即 $P=NP$ 。
- 目前还没有一个NP完全问题有多项式时间算法。

世界七大数学难题

<http://baike.baidu.com/view/253218.htm>



8.3.1 多项式时间变换

设 $L_1 \subseteq \Sigma_1^*$, $L_2 \subseteq \Sigma_2^*$ 是2个语言。所谓语言 L_1 能在多项式时间内变换为语言 L_2 (简记为 $L_1 \propto_p L_2$) 是指存在映身 $f: \Sigma_1^* \rightarrow \Sigma_2^*$, 且 f 满足:

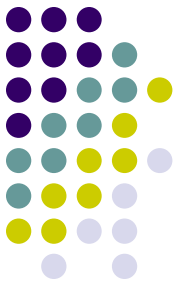
- (1) 有一个计算 f 的多项式时间确定性图灵机;
- (2) 对于所有 $x \in \Sigma_1^*$, $x \in L_1$, 当且仅当 $f(x) \in L_2$ 。

定义: 语言 L 是NP完全的当且仅当

- (1) $L \in \text{NP}$;
- (2) 对于所有 $L' \in \text{NP}$ 有 $L' \propto_p L$ 。

如果有一个语言 L 满足上述性质(2), 但不一定满足性质(1), 则称该语言是**NP难**的。所有NP完全语言构成的语言类称为**NP完全语言类**, 记为**NPC**。

“问题A可约化为问题B”有一个重要的直观意义:
B的时间复杂度高于或者等于**A**的时间复杂度。
也就是说, 问题**A**不比问题**B**难。



8.3.1 多项式时间变换

定理8.6: 设 L 是NP完全的, 则

- (1) $L \in P$ 当且仅当 $P=NP$;
- (2) 若 $L \in_p L_1$, 且 $L_1 \in NP$, 则 L_1 是NP完全的。

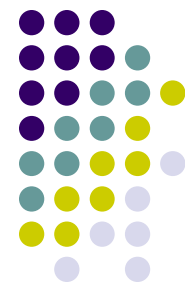
定理的(2)可用来证明问题的NP完全性。但前提是: 要有第一个NP完全问题 L 。



8.3.2 Cook定理

1971年S. Cook发表了“The Complexity of Theorem Proving Procedures”这篇著名论文，1972年R. Karp发表了“Reducibility Among Combinatorial Problems”，从此奠定了NP完全理论的基础。NP完全理论指出在NP类中有一些问题具有以下性质：若其中一个问题获得多项式算法，则这一类问题就全部获得了多项式算法；反之，若能证明其中一个问题是多项式时间内不可解的，则这一类问题就全部是多项式时间内不可解的。这类问题将称之为NP完全问题。NP完全理论不打算找出这一类问题的算法，仅仅着眼于证明这一类问题的等价性，即证明它们的难度相当。

NP完全理论还很年轻，有许多问题等待人们研究。例如，“ $NP=P$ ”还是相反“ P 是NP的真子集”。这个问题是当代计算机科学中的一大难题。



8.3.2 Cook定理

定理8-7 (Cook定理): 布尔表达式的可满足性问题SAT是NP完全的。

证明: SAT的一个实例是 k 个布尔变量 $x_1, \dots, x_k$ 的 m 个布尔表达式 $A_1, \dots, A_m$ 若存在各布尔变量 x_i ($1 \leq i \leq k$)的0, 1赋值, 使每个布尔表达式 A_i ($1 \leq i \leq m$)都取值1, 则称布尔表达式 $A_1 A_2 \dots A_m$ 是可满足的。

SAT $\in$ NP是很明显的。对于任给的布尔变量 $x_1, \dots, x_k$ 的0, 1赋值, 容易在多项式时间内验证相应的 $A_1 A_2 \dots A_m$ 的取值是否均为1。因此, SAT $\in$ NP。

现在只要证明对任意的 $L \in \text{NP}$ 有 $L \leq_p \text{SAT}$ 即可。

(详细证明见书本P307~310)



Cook定理的重要意义

Cook定理是NP完全理论中非常重要得一个定理，这是因为：

首先，它以实例说明NP完全问题是存在的；

其次，它给出了判断某一其它NP问题（更确切地说是表现为这一类问题的语言） L_0 是否是NP完全的捷径，即只须证明：

$$\text{SAT} \leq_M^P L_0$$

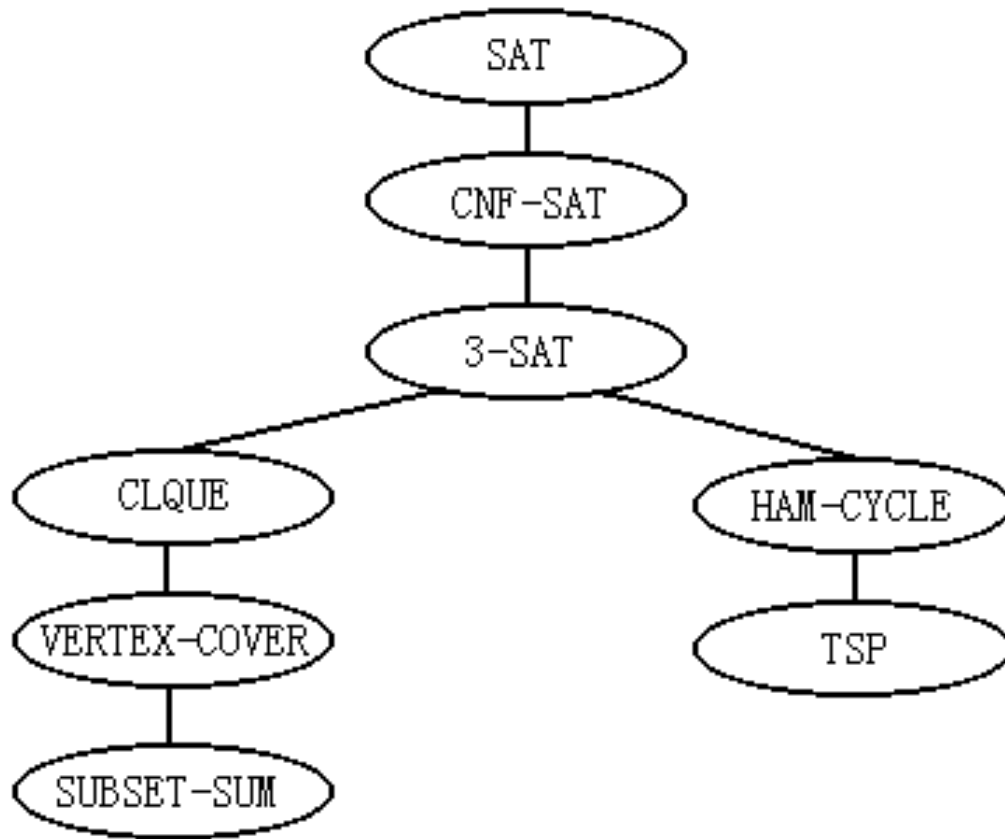
这是因为：所有 $L \in \text{NP}$, $L \leq_M^P \text{SAT}$ ，再由上式和多项式变换的传递性可得：

$$L \leq_M^P L_0$$

一旦证明了 L_0 是NP完全的，则它又可发挥SAT同样的作用。



8.4 一些典型的NP完全问题



部分NP完全问题树

基于SAT，逐渐地生成一棵以SAT为树根的NP完全问题树，其中每个节点代表一个NP完全问题，该问题可在多项式时间内变换为其任意子节点表示的问题。目前，这颗树已有几千个节点，且在继续生长。

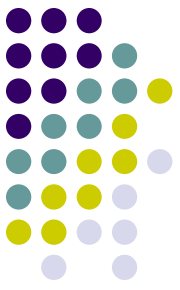


8.4.1 合取范式的可满足性问题 (CNF-SAT)

问题描述： 给定一个合取范式 α ，判定它是否可满足。

如果一个布尔表达式是一些因子和之积，则称之为合取范式，简称CNF (Conjunctive Normal Form)。这里的因子是变量 x 或 $\bar{x}$ 。例如： $(x_1 + x_2)(x_2 + x_3)(\bar{x}_1 + \bar{x}_2 + x_3)$ 就是一个合取范式，而 $x_1x_2 + x_3$ 就不是合取范式。

要证明 $\text{CNF-SAT} \in \text{NPC}$ ，只要证明在Cook定理中定义的布尔表达式 $A, \dots, G$ 或者已是合取范式，或者有的虽然不是合取范式，但可以用布尔代数中的变换方法将它们化成合取范式，而且合取范式的长度与原表达式的长度只差一个常数因子。

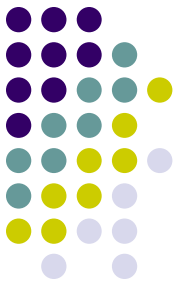


8.4.2 3元合取范式的可满足性问题 (3-SAT)

问题描述： 给定一个3元合取范式 α ，判定它是否可满足。

证明思路：

3-SAT $\in$ NP是显而易见的。为了证明3-SAT $\in$ NPC，只要证明CNF-SAT $\propto_p$ 3-SAT，即合取范式的可满足性问题可在多项式时间内变换为3-SAT。



8.4.3 团问题CLIQUE

问题描述： 给定一个无向图 $G=(V, E)$ 和一个正整数 k ，判定图 G 是否包含一个 k 团，即是否存在， $V' \subseteq V$ ， $|V'|=k$ ，且对任意 $u, w \in V'$ 有 $(u, w) \in E$ 。

证明思路：

已经知道 $\text{CLIQUE} \in \text{NP}$ 。通过 $3\text{-SAT} \leq_p \text{CLIQUE}$ 来证明 CLIQUE 是NP难的，从而证明团问题是NP完全的。



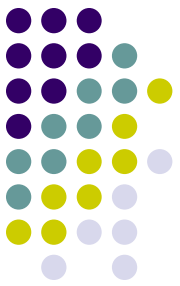
8.4.4 顶点覆盖问题 (VERTEX-COVER)

问题描述： 给定一个无向图 $G=(V, E)$ 和一个正整数 k ，判定是否存在 $V' \subseteq V$ ， $|V'|=k$ ，使得对于任意 $(u, v) \in E$ 有 $u \in V'$ 或 $v \in V'$ 。如果存在这样的 V' ，就称 V' 为图 G 的一个大小为 k 顶点覆盖。

证明思路：

首先， $\text{VERTEX-COVER} \in \text{NP}$ 。因为对于给定的图 G 和正整数 k 以及一个“证书” V' ，验证 $|V'|=k$ ，然后对每条边 $(u, v) \in E$ ，检查是否有 $u \in V'$ 或 $v \in V'$ ，显然可在多项式时间内完成。

其次，通过 $\text{CLIQUE} \propto_p \text{VERTEX-COVER}$ 来证明顶点覆盖问题是NP难的。



8.4.5 子集和问题

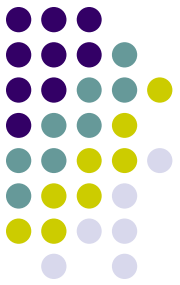
(SUBSET-SUM)

问题描述： 给定整数集合 S 和一个整数 t ，判定是否存在 S 的一个子集 $S' \subseteq S$ ，使得 S' 中整数的和为 t 。例如，若 $S = \{1, 4, 16, 64, 256, 1040, 1041, 1093, 1284, 1344\}$ 且 $t = 3754$ ，则子集 $S' = \{1, 16, 64, 256, 1040, 1093, 1284\}$ 是一个解。

证明思路：

首先，对于子集和问题的一个实例 $\langle S, t \rangle$ ，给定一个“证书” S' ，要验证 $t = \sum_{i \in S'} i$ 是否成立，显然可在多项式时间内完成。因此， $\text{SUBSET-SUM} \in \text{NP}$ ；

其次，证明 $\text{VERTEX-COVER} \leq_p \text{SUBSET-SUM}$ 。

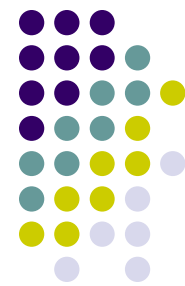


8.4.6 哈密顿回路问题 (HAM-CYCLE)

问题描述： 给定无向图 $G=(V, E)$ ，判定其是否含有一哈密顿回路。

证明思路：

首先，已知哈密顿回路问题是一个NP类问题。
其次，通过证明 $3\text{-SAT} \propto_p \text{HAM-CYCLE}$ ，
得出： $\text{HAM-CYCLE} \in \text{NPC}$ 。



8.4.7 旅行售货员问题TSP

问题描述： 给定一个无向完全图 $G=(V, E)$ 及定义在 $V \times V$ 上的一个费用函数 c 和一个整数 k ，判定 G 是否存在经过 V 中各顶点恰好一次的回路，使得该回路的费用不超过 k 。

首先，给定TSP的一个实例 (G, c, k) ，和一个由 n 个顶点组成的顶点序列。验证算法要验证这 n 个顶点组成的序列是图 G 的一条回路，且经过每个顶点一次。另外，将每条边的费用加起来，并验证所得的和不超过 k 。这个过程显然可在多项式时间内完成，即 $TSP \in NP$ 。

其次，旅行售货员问题与哈密顿回路问题有着密切的联系。哈密顿回路问题可在多项式时间内变换为旅行售货员问题。即 $HAM-CYCLE \propto_p TSP$ 。从而，旅行售货员问题是NP难的。

因此， $TSP \in NPC$ 。

本周任务

- 8.1, 8.2

